

Recuperación UP05 – RA3 - JUnit, trazas y casos de prueba

OBJETIVO DE LA TAREA

En esta práctica deberás:

- Crear pruebas unitarias con **JUnit 5**
- Analizar el comportamiento de un programa
- Diseñar y completar **casos de prueba**
- Detectar y explicar un **error en el código**

Se te proporciona la siguiente clase Java ***EvaluacionFinal.java*** que encontrarás en Aules.

El código contiene al menos un error lógico que deberás detectarlo mediante pruebas y justificarlo.

PARTE 1: PRUEBAS CON JUNIT

Crea una clase de test llamada ***EvaluacionFinalTest.java***

Debes realizar pruebas para comprobar:

Método calcularNotaFinal

- Caso correcto (ejemplo: 6, 7, 8)

Método obtenerResultado

- Suspenso
- Aprobado
- Notable
- Sobresaliente

Casos límite

- 4.9
- 5.0
- 6.9
- 7.0
- 8.9

- 9.0

Otros métodos

- estaAprobado()
- calcularMedia()

Excepciones

- Nota negativa
- Nota mayor que 10
- Array vacío

PARTE 2: DETECCIÓN DEL ERROR

Responde:

1. ¿Qué test falla?
2. ¿Por qué falla?
3. ¿En qué línea está el problema?
4. ¿Cómo se podría corregir?

PARTE 3: CASOS DE PRUEBA

Completa la siguiente tabla:

ID	Método	Entrada	Resultado esperado	Resultado obtenido	¿Correcto?
CP01	calcularNotaFinal	6,7,8	6.9		
CP02	obtenerResultado	4.9	Suspense		
CP03	obtenerResultado	5.0	Aprobado		
CP04	obtenerResultado	6.9	Notable		
CP05	obtenerResultado	7.0	Notable		
CP06	obtenerResultado	8.9	Notable		
CP07	obtenerResultado	9.0	Sobresaliente		

PARTE 4: ANÁLISIS DE TRAZAS

Se proporciona el siguiente código:

```
public class ProcesadorCompras {  
  
    public double procesarCompra(double precio, int cantidad, boolean socio) {  
        System.out.println("Inicio de compra");  
  
        double total = precio * cantidad;  
        System.out.println("Total inicial: " + total);  
  
        if (socio) {  
            total = total * 0.9;  
            System.out.println("Aplicado descuento de socio");  
        }  
  
        if (total > 100) {  
            total = total * 0.95;  
            System.out.println("Aplicado descuento por compra alta");  
        }  
  
        if (cantidad <= 0) {  
            System.out.println("Error: cantidad inválida");  
            throw new IllegalArgumentException("Cantidad inválida");  
        }  
  
        System.out.println("Total final: " + total);  
  
        return total;  
    }  
}
```

Sin ejecutar el programa, indica qué se muestra por consola en los siguientes casos:

Caso 1 → precio = 10, cantidad = 5, socio = false

Caso 2 → precio = 20, cantidad = 10, socio = true

Caso 3 → precio = 50, cantidad = 3, socio = true

Caso 4 → precio = 10, cantidad = 0, socio = false

Preguntas de análisis

Responde:

1. ¿Se calcula el total antes de validar la cantidad?
2. ¿Es correcto ese orden? Justifica.
3. ¿Qué problema puede provocar ese diseño?
4. ¿Dónde debería hacerse la validación?
5. ¿Qué técnica de testing ayuda a detectar este tipo de errores?